Lab experience 1: Introduction to LabVIEW

Chapters refer to 3rd edition of Essick, *Hands-on Introduction to LabVIEW*

Objectives

By the time you start next week's lab exercise (LabVIEW as an oscilloscope) you should be familiar with these concepts and skills:

- The contents of these sections of the text:

  Chapter 2, The While Loop and Waveform Chart
  Chapter 3, The For Loop and Waveform Graph
  Sections 4.1 & 4.2, MathScript Node Basics

- The various data types and the wires that represent them
- The definition and utility of polymorphic functions.
- A few tips for using the LabVIEW interface

Catching up

From their activities in BIOEN 327 and 337, the seniors should be familiar with the LabVIEW features in Chapter 1 (navigating the front panel and block diagram), sections 2.1 - 2.8 (while loops and waveform charts), and section 4.1 with a few exceptions.[1] Specifically, I expect they should be able to create the following:

- The VI shown on pages 66 and 68 (40 and 42 in 2nd edition), which continuously generates a sine signal and plots it on a waveform chart,
- The VI whose block diagram is shown in sections 3.5-3.8, which generates a sine sequence in a For loop and plots it on a waveform graph, and
- A VI that replaces the For Loop with a MathScript node.

Anyone who is new to LabVIEW, or who *feels* as if LabVIEW is new, should introduce themselves properly through the following self-guided exercises:

- The introduction to LabVIEW used in BIOEN 327 (Part 1 only).
- The text and exercises in Chapter 2, Chapter 3 (3.1 - 3.8), and section 4.1.

New concepts for this week

Timers

The standard timers in LV count by milliseconds, which is fine for many real-time operations, but not sufficient for timing the execution of LV functions and loops. For this we can use High Resolution Relative Seconds in the Programming >> Timing menu. If you install the Real-Time Module then you will have access to a microsecond timer (Real Time >> RT Timing >> Tick count) but that module was not specified in the installation instructions for this course.

[1] Three items that were not introduced in the BioE core are the index icon in the while loop (the **i** in the small blue box, introduced on p. 40 in section 2.2), the Scale Legend in the waveform chart or graph (p. 54, Visible Items > Scale Legend), and the "Mechanical action" switch options (p. 60-61).

Data types

Please explore the Control and Indicator Data Types table in the LabVIEW Help >> Contents >> Fundamentals >> Building the Block Diagram >> Concepts >> Block Diagram Objects. We typically use single and double precision floating point, signed and unsigned 8-, 16- and 32-bit integers, enumerated type, Booleans, strings, arrays, clusters, paths, and waveforms. We might use the complex types during Fourier analysis, and we are forced to use the Dynamic Data Type because it is the output format from the Express VI's, even though DDT is unwieldy in most applications.

Tips and notes

The knowledge in this section is not required to use LabVIEW, but it can improve the experience. Its usefulness will be clear after you have used the software a while.

The **Functions palette** may be accessed by right-clicking the block diagram, and the **Controls palette** may be accessed by right-clicking on the front panel. By default, not all of the available functions and controls are shown.

> Bring up the Functions palette; pin it to the desktop; click Customize >> Change visible palettes…; click Select All; click OK. Now you can see the top level of every category in the Functions tree.
>
> Bring up the Controls palette; click Customize >> Change visible palettes. Note that there are four categories at the top of the list: Modern, Silver, System, Classic. These are redundant sets of controls and indicators, which differ only in their appearance. I suggest you make all four of them visible, check out the appearance of each set to determine which one you prefer, then go back and de-select the three that you do not prefer. Depending on the work you do, you might also wish to select the Signal Processing and Control Design & Simulation menus.

By default, when a new control or indicator terminal is placed on the block diagram, it appears as a square icon. This icon contains some pictorial information that indicates the function of the control or indicator, but it consumes more space than necessary. The alternative to an icon is a small rectangle that indicates only the data type. If you wish to use the mini-terminal, right-click the icon and deselect View as icon in the menu. To avoid auto-icons by default, access the Tools menu >> Options >> Block diagram, and deselect Place front panel terminals as icons.

Waveform Charts and Waveform Graphs: Note in chapters 1 and 2 that the single values from the sine function are displayed in a Waveform Chart, while the multiple values produced by the sine function inside a For Loop are displayed in a Waveform Graph. The Chart contains an internal buffer that will accumulate data and display up to 1024 points of a signal.

Axis scaling: A waveform chart can be fed with a combination of time and signal values, or just the signal sequence. If the time values are displayed, they typically appear in a format that is less than optimal (generally more information than we

want, e.g. number of hours elapsed for a trial that lasts a minute or two). One way to avoid this problem is to pass only the signal sequence, and to scale the x-axis within the chart as needed to make it match the original time points. The steps to add this scaling feature into a VI with a *property node,* such that it is adaptable to the time or frequency axis of a plot, are introduced in problem 1.9.b.

New concepts and reading for next week's lab

Polymorphism

Polymorphism is the ability of a function or subroutine to accept a variety of data types as its input while retaining proper functionality. Typically, the more low-level a programming environment is, the less polymorphic its functions are. Sometime during the coming week, read the LabVIEW help section Contents >> Fundamentals >> Building the Block Diagram >> Concepts >> Polymorphic Functions.

Probes

Read section 3.12 of the text, or the appropriate help files, on the use of *probes* for VI debugging purposes.

Clusters

Read section 3.9 of the text, or the appropriate help files, on the bundling of two scalars and one array into a cluster that is then passed to a waveform graph for plotting. The Bundle function is found in the block diagram menu, in Functions >> Programming >> Cluster, Class & Variant.

Week 1 Exercises

1. Loop timing

a) Create a VI that measures how long it takes to execute a While Loop that generates and plots one value of sine per loop, and continuously displays the time values on a waveform **chart**. One good option is to use a Shift Register, initializing it with the High Resolution Relative Seconds function outside the While loop (feeding into the left side of the loop), and include another High-Res-Rel-Sec function inside the loop, which is connected to the Shift Register's terminal on the right side of the While loop. This way, the HRRS value from the previous loop can be subtracted from the current HRRS value and plotted inside the loop. You should include a Wait (ms) function inside the loop, to give a delay of with an input of 1 or 2 msec, then see how much time actually elapses per loop.

b) Modify the VI so that it measures the time needed to execute *N* While Loop iterations, and divide the total time by the number of iterations. It is possible to do this with a For loop, but using the While loop gives you a chance to try out a Greater Than function and a logical OR for loop termination.
Suggestion:

- Delete the Shift Register. Create an Add icon inside the loop and wire the first high-resolution timer (the one that is outside the loop) to it. Make the other addend zero, such that you are just adding zero to the initial time t_0. Then, pass the result out of the while loop. This trick forces one timer to be read before the loop starts and the other timer to be read during the last iteration. Note that the position of the icons on the screen does not determine the order of operations, the flow of data through the functions does. Section 9.3 of the text, on *data dependency*, gives a detailed explanation.
- Create a Subtract icon outside (after) the loop, and wire the t_0 + 0 output to its (–) terminal and the high-precision timer that is inside the loop to its (+) terminal, making sure that the Tunnel Mode for the loop's outlet tunnel is set to Last Value. The starting value is then subtracted from the time of the last loop iteration, after the While loop has stopped.

2. Do Problem 2.7 (Chapter 2, problem 7), which explores the effect of loop operations on computing performance and CPU load. *The following text is copied from the LabVIEW textbook, for those who have not received it yet.*

Open the Windows Task Manager (ctrl+shift+esc in Windows) so you can see the CPU usage indicator and explore the impact of While Loop execution on CPU usage. This feature is provided by the Activity Monitor in MacOS.

(a) Generate (if you have not already) a VI that uses a While Loop to plot a sine wave with a Wait time of 100 msec in each loop. Run the VI and observe the CPU load indicator. What is the usage that results when the VI is running?

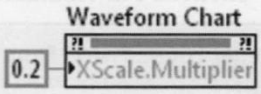
(b) Next, set the Wait time to 1 msec, and run the VI. What is the approximate increase in CPU usage that results when this version of the program is running?

(c) Finally, delete the Wait from the VI's block diagram so the While Loop iterates as fast as possible, then run the program. What is the approximate increase in CPU usage that results when the program is now run?

3. Do problems 9(a) and 9(c) in Chapter 2; the scanned problems are on the following page. These problems involve the use of property nodes, which are introduced in more detail in section 8.2. You might not be able to follow all of Section 8.2 because it uses features that we have not seen yet, but the figures on pp. 338 and 339 do show the menu items that we will use in this exercise.

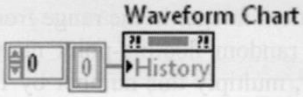
9. A characteristic of the Waveform Chart can be controlled within a program using a **Property Node**.

(a) In this chapter, you set a Waveform Chart's **X-Axis Multiplier** equal to *0.2* using the **Properties** dialog window. Alternately, this property of the Waveform Chart can be set on the block diagram via a Property Node as follows: with **Sine Wave Chart (While Loop)** open, pop up on the Waveform Chart's icon terminal and select **Create>>Property Node>>X Scale>>Offset and Multiplier>>Multiplier**, and then place the resulting Property Node on the block diagram. This Property Node is created as an indicator, hence the small outward-directed black arrow at its right side. Change this icon to a control by popping up on its lower section and selecting **Change To Write**. The black arrow will now be inward directed on its left side. Using **Create>>Constant**, wire a value of *0.2* to the **XScale.Multiplier** property control as shown below.



Run **Sine Wave Chart (While Loop)** and assure yourself that the Property Node is indeed setting the **X-Axis Multiplier** to 0.2.

(b) Program **Sine Wave Chart (While Loop)** so that its Waveform Chart is cleared at the start of each run. The appropriate Property Node is made by selecting **Create>>Property Node>>History Data**. After changing this Property Node to a control by selecting **Change to Write** in its pop-up menu, **Create>>Constant** will produce the needed input called an **Empty Array** as shown below.



To clear the Chart at the start of each program execution (i.e., immediately after the Run button is pressed), should this Property Node be placed inside or outside of the While Loop? Run **Sine Wave Chart (While Loop)** several successive times and demonstrate that the Property Node performs as intended.

(c) Change the background color **Sine Wave Chart (While Loop)**'s Waveform Chart during runtime as follows: place a Property Node within the While Loop using **Create>>Property Node>>Plot Area>>Colors>>BG Color**, and then change this Property Node to a control by selecting **Change To Write** in its pop-up menu. On the front panel, place a **Framed Color Box**, found in **Controls>>Modern>>Numeric**, and then on the block diagram wire its terminal to the Property Node. Run **Sine Wave Chart (While Loop)** and demonstrate that the Waveform Chart's background color can be controlled using the Operating Tool.

General notes & tips (not an assignment):

a) *Charts* collect incoming data in a buffer, so one datum per loop is sufficient. *Graphs* discard all previously received data, so one point per loop is not enough.

b) Controls and indicators may be interchanged by selecting "Change to" on the right-click menu. Note that the appearance usually changes on both the diagram and the front panel.

c) On the front panel, pressing the space bar toggles the mouse pointer between the finger (change input) and arrow (move/size display). On the diagram, the space bar cycles the pointer through the wire, arrow and finger.

d) By default, new controls and indicators show up on the block diagram as squares like this 

. You can make them smaller by right clicking and unchecking "show as icon." Control/indicator icons with bold outlines on the block diagram are represent controls on the front panel, while rectangles with narrow outlines represent indicators.

e) Any VI can be used as a subroutine within a larger VI. Typically these sub-VIs appear as white squares, which can be opened and edited (right click > Open front panel). They perform a variety of procedures and complicated mathematical functions.

f) When complicated instrument drivers are involved, the fastest way to get the program you need is often to modify one of the example VI's. Open Help>>Examples>>I/O Interfaces>>DAQ Examples; the Analog Input and Analog Output lists contain the most useful starting points for our data acquisition hardware.